

Drivechains: BIP 300, BIP 301, and eCash

Agenda

- The two-way peg problem

Agenda

- The two-way peg problem
- BIP 300 hashrate escrows

Agenda

- The two-way peg problem
- BIP 300 hashrate escrows
- BIP 301 blind merged mining

Agenda

- The two-way peg problem
- BIP 300 hashrate escrows
- BIP 301 blind merged mining
- Paul Sztorc's planned eCash fork, CUSF, and mining compatibility

Drivechains

The three bois

BIP 300

Peg state, deposits, and
miner-approved withdrawals

BIP 301

A fee market for committing
sidechain blocks

eCash

A separate L1 fork that plans
to enable both

So far, eCash has attracted far less controversy than BIP 110.

Proposal status

- **BIP 300:** Hashrate Escrows, assigned August 2017, Draft

Proposal status

- **BIP 300:** Hashrate Escrows, assigned August 2017, Draft
- **BIP 301:** Blind Merged Mining, assigned July 2019, Draft

Proposal status

- **BIP 300:** Hashrate Escrows, assigned August 2017, Draft
- **BIP 301:** Blind Merged Mining, assigned July 2019, Draft
- Both are consensus soft-fork proposals

Proposal status

- **BIP 300:** Hashrate Escrows, assigned August 2017, Draft
- **BIP 301:** Blind Merged Mining, assigned July 2019, Draft
- Both are consensus soft-fork proposals
- Neither is active on Bitcoin

Proposal status

- **BIP 300:** Hashrate Escrows, assigned August 2017, Draft
- **BIP 301:** Blind Merged Mining, assigned July 2019, Draft
- Both are consensus soft-fork proposals
- Neither is active on Bitcoin
- Current implementations can enforce them outside Bitcoin Core through CUSF

Terms

- **Sidechain:** a separate ledger with its own validation rules

Terms

- **Sidechain:** a separate ledger with its own validation rules
- **Peg-in:** lock L1 coins and recognize an equivalent balance on the sidechain

Terms

- **Sidechain:** a separate ledger with its own validation rules
- **Peg-in:** lock L1 coins and recognize an equivalent balance on the sidechain
- **Peg-out:** destroy or lock the sidechain balance and release coins on L1

Terms

- **Sidechain:** a separate ledger with its own validation rules
- **Peg-in:** lock L1 coins and recognize an equivalent balance on the sidechain
- **Peg-out:** destroy or lock the sidechain balance and release coins on L1
- **Treasury UTXO:** the single L1 output holding one sidechain's locked funds

Terms

- **Sidechain:** a separate ledger with its own validation rules
- **Peg-in:** lock L1 coins and recognize an equivalent balance on the sidechain
- **Peg-out:** destroy or lock the sidechain balance and release coins on L1
- **Treasury UTXO:** the single L1 output holding one sidechain's locked funds
- **CTIP:** the cached txid and output index of the current treasury UTXO

Terms

- **Sidechain:** a separate ledger with its own validation rules
- **Peg-in:** lock L1 coins and recognize an equivalent balance on the sidechain
- **Peg-out:** destroy or lock the sidechain balance and release coins on L1
- **Treasury UTXO:** the single L1 output holding one sidechain's locked funds
- **CTIP:** the cached txid and output index of the current treasury UTXO
 - Current Treasury Index Pointer

Terms

- **Sidechain:** a separate ledger with its own validation rules
- **Peg-in:** lock L1 coins and recognize an equivalent balance on the sidechain
- **Peg-out:** destroy or lock the sidechain balance and release coins on L1
- **Treasury UTXO:** the single L1 output holding one sidechain's locked funds
- **CTIP:** the cached txid and output index of the current treasury UTXO
 - Current Treasury Index Pointer
- **Withdrawal bundle:** one L1 transaction aggregating many sidechain peg-outs

Terms

- **Sidechain:** a separate ledger with its own validation rules
- **Peg-in:** lock L1 coins and recognize an equivalent balance on the sidechain
- **Peg-out:** destroy or lock the sidechain balance and release coins on L1
- **Treasury UTXO:** the single L1 output holding one sidechain's locked funds
- **CTIP:** the cached txid and output index of the current treasury UTXO
 - Current Treasury Index Pointer
- **Withdrawal bundle:** one L1 transaction aggregating many sidechain peg-outs
- **Blind merged mining:** buying an L1 commitment to an L2 block without asking the L1 miner to validate that L2

A two-way peg



Deposits move value right. Approved withdrawal bundles move value left.

Withdrawal approval by block voting

Federated peg

A known threshold of keys signs withdrawals

Drivechain peg

Future mainchain blocks accumulate approval
for a withdrawal hash

BIP 300 replaces fixed-keyholder approval with approval accumulated across future mainchain block templates.

BIP 300

L1 state

- Up to 256 numbered sidechain slots

L1 state

- Up to 256 numbered sidechain slots
- A sidechain list records each active slot and its current treasury outpoint

L1 state

- Up to 256 numbered sidechain slots
- A sidechain list records each active slot and its current treasury outpoint
- A withdrawal list records candidate bundle hashes, scores, and expiry

L1 state

- Up to 256 numbered sidechain slots
- A sidechain list records each active slot and its current treasury outpoint
- A withdrawal list records candidate bundle hashes, scores, and expiry
- Bitcoin nodes enforcing BIP 300 track this state

L1 state

- Up to 256 numbered sidechain slots
- A sidechain list records each active slot and its current treasury outpoint
- A withdrawal list records candidate bundle hashes, scores, and expiry
- Bitcoin nodes enforcing BIP 300 track this state
- They do not validate the sidechain's transactions or ledger

messages

Sidechain slots

M1 proposes
M2 acknowledges

Withdrawal approval

M3 proposes a bundle
M4 changes its score

Peg movement

M5 deposits
M6 withdraws

Treasury output

Current treasury UTXO

`OP_DRIVECHAIN <1-byte slot> OP_TRUE`

- `OP_NOP5` becomes `OP_DRIVECHAIN` only in this exact script form

Treasury output

Current treasury UTXO

`OP_DRIVECHAIN <1-byte slot> OP_TRUE`

- `OP_NOP5` becomes `OP_DRIVECHAIN` only in this exact script form
- Legacy nodes see a no-op followed by true, so the spend is valid

Treasury output

Current treasury UTXO

`OP_DRIVECHAIN <1-byte slot> OP_TRUE`

- `OP_NOP5` becomes `OP_DRIVECHAIN` only in this exact script form
- Legacy nodes see a no-op followed by true, so the spend is valid
- Enforcing nodes permit only M5 or an approved M6

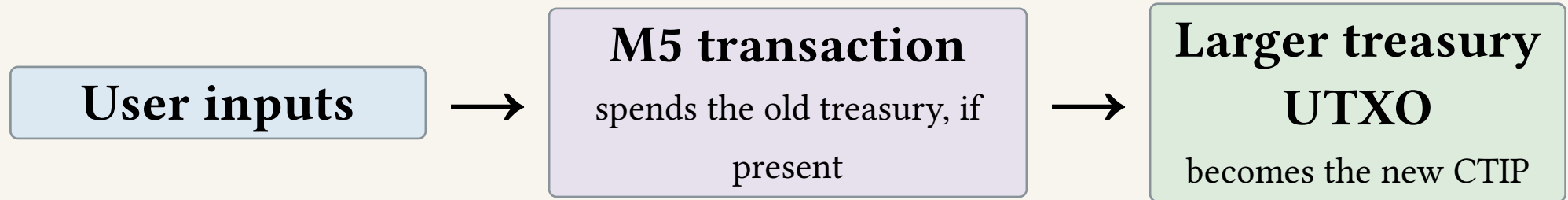
Treasury output

Current treasury UTXO

`OP_DRIVECHAIN <1-byte slot> OP_TRUE`

- `OP_NOP5` becomes `OP_DRIVECHAIN` only in this exact script form
- Legacy nodes see a no-op followed by true, so the spend is valid
- Enforcing nodes permit only M5 or an approved M6
- Every spend recreates one treasury UTXO for the slot

Deposit path



The sidechain observes the deposit and applies its own rules for crediting the recipient.

Withdrawal path

1. Sidechain users request peg-outs
- 2.
- 3.
- 4.
- 5.
- 6.

Withdrawal path

1. Sidechain users request peg-outs
2. Sidechain software aggregates up to 6,000 payments into one M6 transaction
- 3.
- 4.
- 5.
- 6.

Withdrawal path

1. Sidechain users request peg-outs
2. Sidechain software aggregates up to 6,000 payments into one M6 transaction
3. M3 places the blinded M6 hash in the mainchain withdrawal list
- 4.
- 5.
- 6.

Withdrawal path

1. Sidechain users request peg-outs
2. Sidechain software aggregates up to 6,000 payments into one M6 transaction
3. M3 places the blinded M6 hash in the mainchain withdrawal list
4. Mainchain blocks change that candidate's score through M4
- 5.
- 6.

Withdrawal path

1. Sidechain users request peg-outs
2. Sidechain software aggregates up to 6,000 payments into one M6 transaction
3. M3 places the blinded M6 hash in the mainchain withdrawal list
4. Mainchain blocks change that candidate's score through M4
5. A score of 13,150 makes the corresponding M6 eligible
- 6.

Withdrawal path

1. Sidechain users request peg-outs
2. Sidechain software aggregates up to 6,000 payments into one M6 transaction
3. M3 places the blinded M6 hash in the mainchain withdrawal list
4. Mainchain blocks change that candidate's score through M4
5. A score of 13,150 makes the corresponding M6 eligible
6. M6 pays users and recreates the treasury UTXO with the remaining balance

Vote arithmetic

Selected

+1

Competing

-1

No M4

0

Alarm

-1 to all

- Required score: **13,150**

Vote arithmetic

Selected

+1

Competing

-1

No M4

0

Alarm

-1 to all

- Required score: **13,150**
- Candidate lifetime: **26,300 blocks**

Vote arithmetic

Selected

+1

Competing

-1

No M4

0

Alarm

-1 to all

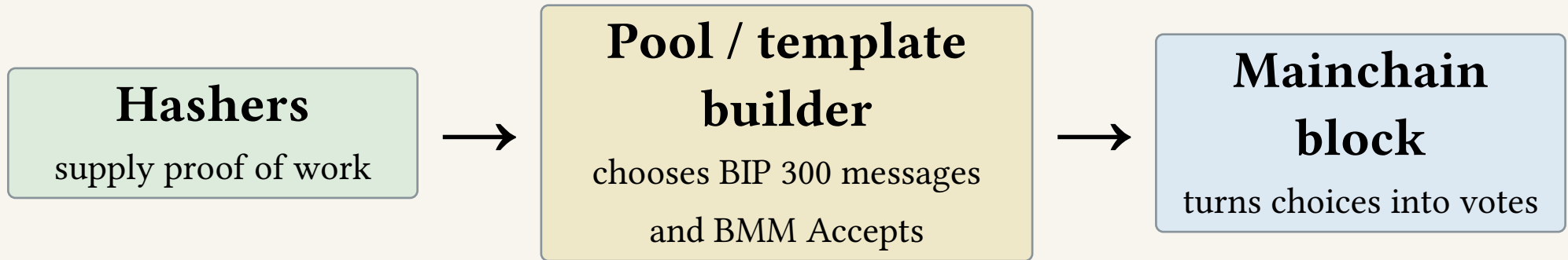
- Required score: **13,150**
- Candidate lifetime: **26,300 blocks**
- Fastest approval is about 91 days; the full window is about 183 days

Vote arithmetic

Selected +1	Competing -1	No M4 0	Alarm -1 to all
-----------------------	------------------------	-------------------	---------------------------

- Required score: **13,150**
- Candidate lifetime: **26,300 blocks**
- Fastest approval is about 91 days; the full window is about 183 days
- Downvotes increase the number of positive blocks required

Block templates cast the votes



Under ordinary pooled mining, the pool votes. An ASIC owner's fraction of hashrate is not independently represented on-chain.

Withdrawal approval properties

- A withdrawal hash remains in the candidate set while its score changes
 - A withdrawal cannot reach approval in one block

Withdrawal approval properties

- A withdrawal hash remains in the candidate set while its score changes
 - A withdrawal cannot reach approval in one block
- L1 does not verify sidechain state or a fraud proof
 - Sufficient approving templates can make a sidechain-invalid withdrawal eligible

Withdrawal approval properties

- A withdrawal hash remains in the candidate set while its score changes
 - A withdrawal cannot reach approval in one block
- L1 does not verify sidechain state or a fraud proof
 - Sufficient approving templates can make a sidechain-invalid withdrawal eligible
- Miners may abstain, downvote, or omit the final M6
 - A valid peg-out may remain pending

Withdrawal approval properties

- A withdrawal hash remains in the candidate set while its score changes
 - A withdrawal cannot reach approval in one block
- L1 does not verify sidechain state or a fraud proof
 - Sufficient approving templates can make a sidechain-invalid withdrawal eligible
- Miners may abstain, downvote, or omit the final M6
 - A valid peg-out may remain pending
- Monitoring can identify a sidechain-invalid bundle
 - Identification does not change its score or block eligibility

Bridge, not block production

- BIP 300 defines sidechain slots and the two-way peg

Bridge, not block production

- BIP 300 defines sidechain slots and the two-way peg
- It does not select or order sidechain blocks

Bridge, not block production

- BIP 300 defines sidechain slots and the two-way peg
- It does not select or order sidechain blocks
- Doesn't require one particular sidechain consensus either

Bridge, not block production

- BIP 300 defines sidechain slots and the two-way peg
- It does not select or order sidechain blocks
- Doesn't require one particular sidechain consensus either
- BIP 301 supplies the proposed block-production market

BIP 301

Classical and blind merged mining

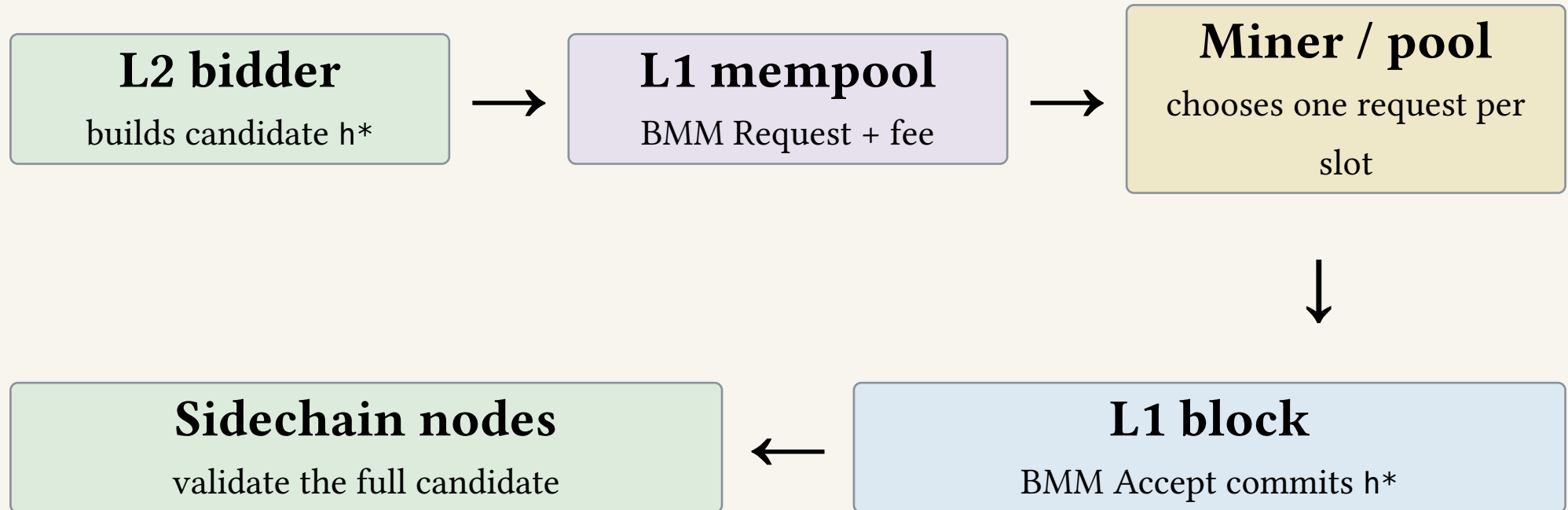
Classical merged mining

Miner runs the other chain's node, validates its block, and receives its native coin

Blind merged mining

Sidechain user builds the block and bids an L1 fee for the miner's commitment

BMM actors



One BMM round

1. A sidechain node constructs a candidate block and its hash, h^* (hash of proposed sidechain block)
- 2.
- 3.
- 4.
- 5.

One BMM round

1. A sidechain node constructs a candidate block and its hash, h^* (hash of proposed sidechain block)
2. The bidder broadcasts an L1 transaction naming the slot, h^* , and previous L1 block
- 3.
- 4.
- 5.

One BMM round

1. A sidechain node constructs a candidate block and its hash, h^* (hash of proposed sidechain block)
2. The bidder broadcasts an L1 transaction naming the slot, h^* , and previous L1 block
3. Its transaction fee is the bid to the mainchain miner
- 4.
- 5.

One BMM round

1. A sidechain node constructs a candidate block and its hash, h^* (hash of proposed sidechain block)
2. The bidder broadcasts an L1 transaction naming the slot, h^* , and previous L1 block
3. Its transaction fee is the bid to the mainchain miner
4. The miner includes at most one matching BMM Accept per slot in the coinbase
- 5.

One BMM round

1. A sidechain node constructs a candidate block and its hash, h^* (hash of proposed sidechain block)
2. The bidder broadcasts an L1 transaction naming the slot, h^* , and previous L1 block
3. Its transaction fee is the bid to the mainchain miner
4. The miner includes at most one matching BMM Accept per slot in the coinbase
5. Sidechain nodes validate the candidate and either connect it or reject it

Blindness boundary

- Mainchain miner sees the sidechain slot, h^* , and L1 fee

Blindness boundary

- Mainchain miner sees the sidechain slot, h^* , and L1 fee
- No need to the sidechain node or inspect sidechain transactions

Blindness boundary

- Mainchain miner sees the sidechain slot, h^* , and L1 fee
- No need to the sidechain node or inspect sidechain transactions
- Sidechain nodes still apply every sidechain validation rule

Blindness boundary

- Mainchain miner sees the sidechain slot, h^* , and L1 fee
- No need to the sidechain node or inspect sidechain transactions
- Sidechain nodes still apply every sidechain validation rule
- If the winning candidate is invalid, no sidechain block is added for that round

Blindness boundary

- Mainchain miner sees the sidechain slot, h^* , and L1 fee
- No need to the sidechain node or inspect sidechain transactions
- Sidechain nodes still apply every sidechain validation rule
- If the winning candidate is invalid, no sidechain block is added for that round
- The miner still receives the L1 fee

BIP 300 and 301 together

BIP 301

Which candidate becomes the next
sidechain block?

BIP 300

When may the mainchain treasury release
funds?

blind mining commits the L2 block sequence, BIP 300 controls eligibility
of the L1 withdrawal tx

Outcomes in selected cases

- A template builder ignores BMM requests
 - That sidechain receives no anchored block for the round

Outcomes in selected cases

- A template builder ignores BMM requests
 - That sidechain receives no anchored block for the round
- An invalid candidate wins the bid
 - Sidechain nodes reject it; the bidder loses the L1 fee

Outcomes in selected cases

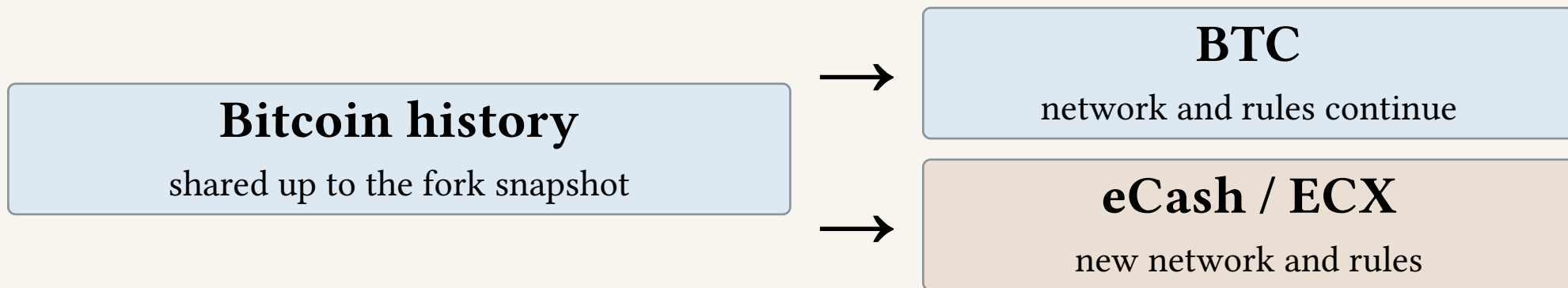
- A template builder ignores BMM requests
 - That sidechain receives no anchored block for the round
- An invalid candidate wins the bid
 - Sidechain nodes reject it; the bidder loses the L1 fee
- The mainchain reorganizes
 - Sidechain commitments reorganize with it

Outcomes in selected cases

- A template builder ignores BMM requests
 - That sidechain receives no anchored block for the round
- An invalid candidate wins the bid
 - Sidechain nodes reject it; the bidder loses the L1 fee
- The mainchain reorganizes
 - Sidechain commitments reorganize with it
- An invalid withdrawal accumulates enough score
 - BIP 300 permits the treasury spend; BIP 301 does not prevent it

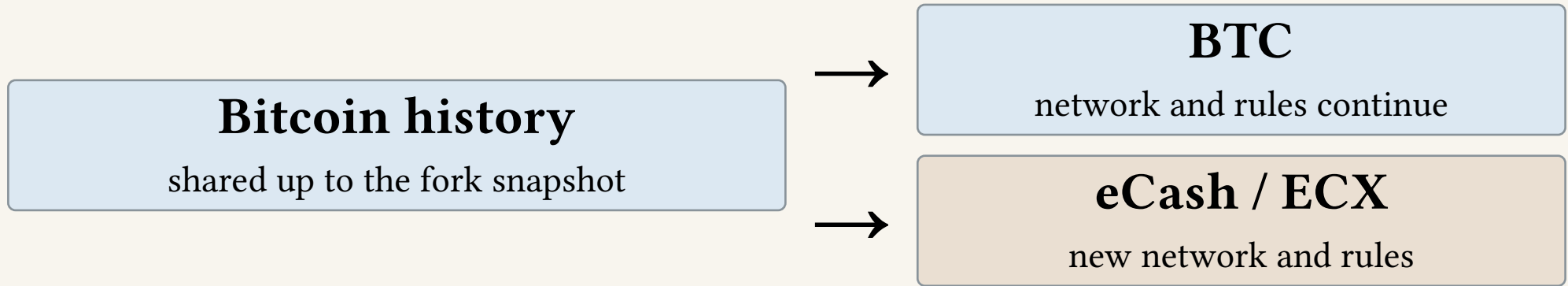
eCash

A planned Bitcoin fork



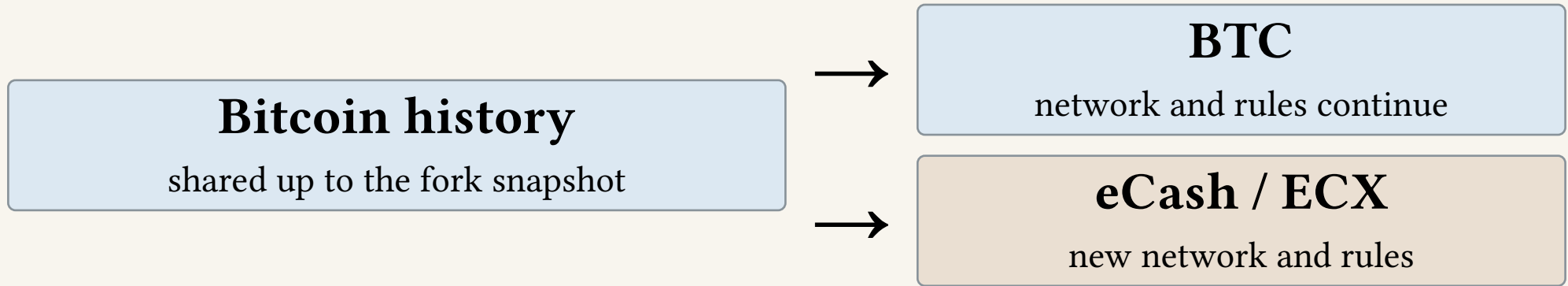
- Planned target: BTC block **963,648**

A planned Bitcoin fork



- Planned target: BTC block **963,648**
- Current estimate: **22 August 2026**

A planned Bitcoin fork



- Planned target: BTC block **963,648**
- Current estimate: **22 August 2026**
- Planned SHA-256d chain; it does not change BTC consensus rules or the BTC ledger

Core Untouched Soft Fork

- CUSF keeps the added validation rules in a separate program
 - The Core-compatible node retains its existing validation code

Core Untouched Soft Fork

- CUSF keeps the added validation rules in a separate program
 - The Core-compatible node retains its existing validation code
- The enforcer reads blocks through RPC and ZMQ
 - If a block violates the added rules, it calls `invalidateblock`

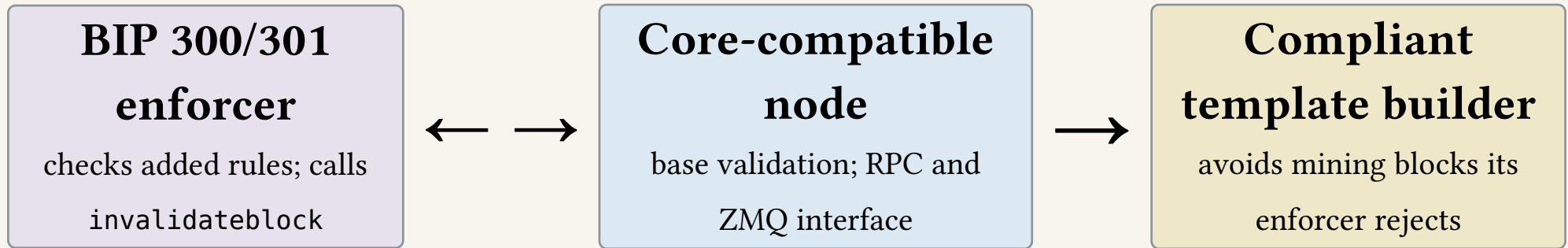
Core Untouched Soft Fork

- CUSF keeps the added validation rules in a separate program
 - The Core-compatible node retains its existing validation code
- The enforcer reads blocks through RPC and ZMQ
 - If a block violates the added rules, it calls `invalidateblock`
- A miner using CUSF builds templates that satisfy the same added rules
 - Otherwise its own enforcer rejects the resulting block

Core Untouched Soft Fork

- CUSF keeps the added validation rules in a separate program
 - The Core-compatible node retains its existing validation code
- The enforcer reads blocks through RPC and ZMQ
 - If a block violates the added rules, it calls `invalidateblock`
- A miner using CUSF builds templates that satisfy the same added rules
 - Otherwise its own enforcer rejects the resulting block
- Nodes without the enforcer continue applying their existing rules
 - CUSF is still a soft fork: enforcing nodes accept a subset of base-valid blocks

CUSF stack



CUSF leaves the added rules outside the node's source tree. Operators running the enforcer treat those rules as consensus.

Fork rules

Fork bootstrap

One-time minimum difficulty

New network magic, seeds, and name

Optional transaction replay-control byte

CUSF-enforced rules

BIP 300 and BIP 301

Planned 400 kB block cap

The project intends to keep the remaining L1 code close to upstream Bitcoin Core.

Ledger distribution

- Ordinary pre-fork BTC UTXOs are mirrored as ECX at 1:1

Ledger distribution

- Ordinary pre-fork BTC UTXOs are mirrored as ECX at 1:1
- The corresponding private keys control the coins on both chains

Ledger distribution

- Ordinary pre-fork BTC UTXOs are mirrored as ECX at 1:1
- The corresponding private keys control the coins on both chains
- Spending ECX does not spend BTC, once replay protection is used

Ledger distribution

- Ordinary pre-fork BTC UTXOs are mirrored as ECX at 1:1
- The corresponding private keys control the coins on both chains
- Spending ECX does not spend BTC, once replay protection is used
- The Satoshi Half-Airdrop reallocates about 550,000 ECX associated with “Patoshi” outputs

Ledger distribution

- Ordinary pre-fork BTC UTXOs are mirrored as ECX at 1:1
- The corresponding private keys control the coins on both chains
- Spending ECX does not spend BTC, once replay protection is used
- The Satoshi Half-Airdrop reallocates about 550,000 ECX associated with “Patoshi” outputs
- That reallocation changes only the new chain’s ledger

Braiins Hashpower and ECX

- Similar to BCash
- Existing BTC delivery does not change
 - eCash is a separate chain with separate pool jobs and payouts

Braiins Hashpower and ECX

- Similar to BCash
- Existing BTC delivery does not change
 - eCash is a separate chain with separate pool jobs and payouts
- Hashpower can route bought SHA-256 hashrate to a custom destination pool
 - The pool must speak Stratum V1, report `extranonce2_size` ≥ 7 , and accept `mining.authorize`

Brains Hashpower and ECX

- Similar to BCash
- Existing BTC delivery does not change
 - eCash is a separate chain with separate pool jobs and payouts
- Hashpower can route bought SHA-256 hashrate to a custom destination pool
 - The pool must speak Stratum V1, report `extranonce2_size` ≥ 7 , and accept `mining.authorize`
- The ECX pool builds the templates
 - It chooses BIP 300 votes and BMM Accepts; Hashpower routes the proof of work

Braiins Hashpower and ECX

- Similar to BCash
- Existing BTC delivery does not change
 - eCash is a separate chain with separate pool jobs and payouts
- Hashpower can route bought SHA-256 hashrate to a custom destination pool
 - The pool must speak Stratum V1, report `extranonce2_size` ≥ 7 , and accept `mining.authorize`
- The ECX pool builds the templates
 - It chooses BIP 300 votes and BMM Accepts; Hashpower routes the proof of work
- Hashpower does not need to interpret BIP 300 or BIP 301
 - Under the published interface, it can mine ECX through a compatible ECX pool

Fork-day

- ECX starts its first 2,016 blocks at minimum network difficulty (maybe)
 - The destination pool and mining firmware must accept jobs at that target

Fork-day

- ECX starts its first 2,016 blocks at minimum network difficulty (maybe)
 - The destination pool and mining firmware must accept jobs at that target
- SoloFork tested a Braiins OS miner against its L2L signet endpoint
 - It reported a BOSminer assertion when share difficulty exceeded network difficulty

Fork-day

- ECX starts its first 2,016 blocks at minimum network difficulty (maybe)
 - The destination pool and mining firmware must accept jobs at that target
- SoloFork tested a Braiins OS miner against its L2L signet endpoint
 - It reported a BOSminer assertion when share difficulty exceeded network difficulty
 - <https://solofork.com/blog/bosminer-low-diff-panic>

Public pool activity

- Major pool accounts
 - No eCash-specific announcement found from AntPool, F2Pool, ViaBTC, Foundry, Braiins, SpiderPool, or Luxor as of 3 August 2026

Public pool activity

- Major pool accounts
 - No eCash-specific announcement found from AntPool, F2Pool, ViaBTC, Foundry, Braiins, SpiderPool, or Luxor as of 3 August 2026
- SoloFork
 - Independent CKpool-based ECX solo pool
 - Uses an L2L signet endpoint for Stratum integration testing
 - Plans to switch its public endpoint to ECX mainnet at launch
 - A found block pays the miner directly, minus a 2% operator output

Public pool activity

- Major pool accounts
 - No eCash-specific announcement found from AntPool, F2Pool, ViaBTC, Foundry, Braiins, SpiderPool, or Luxor as of 3 August 2026
- SoloFork
 - Independent CKpool-based ECX solo pool
 - Uses an L2L signet endpoint for Stratum integration testing
 - Plans to switch its public endpoint to ECX mainnet at launch
 - A found block pays the miner directly, minus a 2% operator output
- LayerTwo Labs also publishes SimplePool
 - It is Stratum V1 pool software, not really a public commitment from a pool

Planned sidechains

- **Thunder**: high-throughput payments
- **zSide**: Zcash-style privacy
- **BitNames**: identity and DNS
- **BitAssets**: issued assets
- **Truthcoin**: prediction markets
- **CoinShift**: decentralized exchange
- **Photon**: post-quantum signatures

eCash

Q&A